OpenZeppelin | security

OpenZeppelin Fixed-Point Math Library Audit

Sui

April 7, 2026

This document has been prepared solely for the named client and it may not be relied upon by any third parties. The content in this document is provided for informational purposes only and does not constitute professional advice of any kind. Security assessments are limited by factors such as scope, time, and techniques used and therefore this document does not constitute a certification, guarantee, or warranty regarding the proper functioning of any system or the absence of security vulnerabilities.

Copyright © 2026 Zeppelin Group Ltd.

Table of Contents

Table of Contents 2

Summary 3

Scope 4

System Overview 5

UD30x9 5

SD29x9 5

Type Conversions 6

Security Model and Trust Assumptions 6

Medium Severity 8

M-01 Missing Scaled Conversion Functions Between Integer and Fixed-Point Types 8

M-02 Bitwise Operations on Signed Fixed-Point Type Expose Raw Two's Complement Encoding 8

Low Severity 10

L-01 SD29x9 pow Uses Asymmetric Boundary Validation 10

L-02 Unsigned Fixed-Point Left Shift Silently Truncates on Overflow 10

L-03 Signed mod Function Name Does Not Reflect Truncated Remainder Semantics 11

L-04 Fixed-Point pow Uses Linear Exponentiation, Accumulating Unnecessary Rounding Error 11

L-05 Decompose Is Not Publicly Exposed, Forcing Users to Validate Independently 12

L-06 Missing Rounding-Direction Variants for Multiplication and Division 13

Notes & Additional Information 13

N-01 Redundant Bitmask in SD29x9::lshift 13

N-02 Division-by-Zero Check in mod and div Relies on VM Runtime Abort 14

N-03 UD30x9::sub Uses Overflow Error Constant for Underflow Condition 14

N-04 Inconsistent SCALE Constant Type Between SD29x9 Modules 15

N-05 Missing Rounding Direction Documentation in SD29x9 Arithmetic Functions 15

Conclusion 17

Appendix 18

Issue Classification 18

Summary

Type	Library
Timeline	From 2026-03-25 To 2026-03-27
Languages	Move

Total Issues	13 (11 resolved, 1 partially resolved)
Critical Severity Issues	0 (0 resolved)
High Severity Issues	0 (0 resolved)
Medium Severity Issues	2 (2 resolved)
Low Severity Issues	6 (4 resolved, 1 partially resolved)
Notes & Additional Information	5 (5 resolved)
Client Reported Issues	0 (0 resolved)

Scope

OpenZeppelin performed an audit of the `OpenZeppelin/contracts-sui` repository at the `7cfc07c` commit.

In scope were the following files:

```
math/fixed_point/sources
├── casting
│   └── u128.move
├── sd29x9
│   ├── sd29x9.move
│   └── sd29x9_base.move
└── ud30x9
    ├── ud30x9.move
    └── ud30x9_base.move
```

System Overview

The `openzeppelin_fp_math` package is a fixed-point arithmetic library for Sui Move. It provides two fixed-point types with 9 decimal places of precision: an unsigned type (`UD30x9`) and a signed type (`SD29x9`). The 9-decimal scale factor aligns with Sui's native coin precision, making the library a building block for DeFi protocols that need exact decimal arithmetic without floating-point approximation. Both types are stored in a single `u128` and expose a broad API covering arithmetic, comparison, rounding, bitwise, and cross-type conversion operations.

UD30x9

`UD30x9` is the unsigned fixed-point type. It wraps a raw `u128` value scaled by `10^9`, where the lower 9 decimal digits represent the fractional part. The type definition and constructors reside in `ud30x9.move`, while all arithmetic and utility operations are in `ud30x9_base.move`.

The public API includes:

- **Construction and casting**: `zero`, `one`, and `max` return fixed boundary values. `wrap` and `unwrap` convert between the raw `u128` representation and the `UD30x9` type.
- **Arithmetic**: `add`, `sub`, `mul`, `div`, `mod`, and `pow`. Multiplication and division use `u256` intermediates to preserve precision before rescaling. Checked operations abort on overflow; unchecked variants (`unchecked_add`, `unchecked_sub`) perform wrapping arithmetic modulo `2^128`.
- **Comparison and rounding**: Full ordering (`eq`, `lt`, `gt`, etc.) and integer rounding via `ceil` and `floor`.
- **Bitwise**: `and`, `or`, `xor`, `not`, `lshift`, and `rshift` operate directly on the underlying `u128` bits.

SD29x9

`SD29x9` is the signed fixed-point type. It stores values in two's complement encoding within a `u128`, where bit 127 serves as the sign bit. The representable range spans from `-2^127`

(`min`) to `2^127 - 1` (`max`). The type definition resides in `sd29x9.move` and arithmetic operations in `sd29x9_base.move`.

Since Sui Move has no native signed integer type, the library implements sign handling internally. Arithmetic functions decompose the raw `u128` bits into a sign-magnitude representation (`Components`) via a private `decompose` helper, perform the operation on unsigned magnitudes stored as `u256`, and recompose the result through `wrap_components`, which enforces that the magnitude fits within the signed range.

The `SD29x9` API mirrors `UD30x9` and additionally provides:

- **Sign-aware operations**: `negate` and `abs`, both of which abort on the minimum value (`-2^127`) since its positive counterpart is not representable.
- **Arithmetic right shift**: `rshift` preserves the sign by sign-extending negative values, unlike the logical right shift in `UD30x9`.
- **Bitwise operations**: `and`, `or`, `xor`, `not`, and `lshift` operate on the raw two's complement bit pattern via `from_bits`, bypassing the sign-magnitude decomposition used by arithmetic functions.

Type Conversions

The library provides bidirectional conversions between the two fixed-point types. `into_UD30x9` converts a non-negative `SD29x9` to `UD30x9`, aborting on negative input. `into_SD29x9` converts a `UD30x9` to `SD29x9`, aborting if the value exceeds `2^127 - 1`. Both have fallible variants (`try_into_UD30x9`, `try_into_SD29x9`) that return `Option` instead of aborting. A separate `casting_u128` module provides direct `u128`-to-`UD30x9` conversion.

Security Model and Trust Assumptions

The library is a stateless, pure-function package with no on-chain storage, no privileged roles, and no administrative capabilities. Its security properties depend entirely on the correctness of its arithmetic implementations and the clarity of its API contracts. Since the library is intended

as a shared primitive for DeFi integrations, correctness failures can propagate into any protocol that depends on it.

- Both types use `wrap` and `unwrap` to move between raw `u128` values and the fixed-point types. These functions operate on the scaled internal representation, not on human-readable integers. Integrators are expected to understand that a raw value of `5` represents `0.000000005`, not `5`, and to apply the `10^9` scale factor manually when constructing or interpreting values.
- Checked arithmetic operations (`add`, `sub`, `mul`, `div`, `pow`) abort on overflow via `assert!` statements with library-defined error constants. Unchecked variants (`unchecked_add`, `unchecked_sub`) silently wrap, placing the responsibility for overflow safety on the caller.
- Multiplication and division in both types use `u256` intermediates to avoid precision loss during rescaling. For `SD29x9`, the intermediate magnitudes (up to `~2^127 x 2^127 = 2^254`) and for `UD30x9` (up to `~2^128 x 2^128 ≈ 2^256`) fit within the `u256` range, so no intermediate overflow occurs under normal operation.
- The library has no external dependencies beyond the Sui Move standard library and is self-contained within the `openzeppelin_fp_math` package namespace.

Medium Severity

M-01 Missing Scaled Conversion Functions Between Integer and Fixed-Point Types

The `SD29x9` and `UD30x9` fixed-point types represent real numbers by storing a `u128` value scaled by `10^9`. The library provides `wrap` and `unwrap` functions for both types, as well as a `casting_u128::into_UD30x9` helper. However, all of these operate on raw bit representations and do not apply the `10^9` scaling factor. The only function that internally accounts for scaling is `one()`, which returns `SCALE` to represent the integer `1`.

There are no public functions to convert a plain integer (e.g., `5`) into its correctly scaled fixed-point representation (`5 * 10^9`), nor to extract the integer part from a fixed-point value by dividing out the scale. This is a fundamental operation present in virtually all established fixed-point math libraries. Without it, integrators must manually multiply or divide by `10^9` when constructing or interpreting values, which is error-prone. Passing an unscaled integer directly to `wrap` , a natural but incorrect usage , would silently produce a value off by a factor of `10^9`, leading to severe arithmetic errors in any downstream computation.

Consider adding `from_int` and `to_int` (or equivalently named) conversion functions for both `UD30x9` and `SD29x9` that apply the appropriate scaling. A `from_int` function would multiply the input by `SCALE` and check for overflow, while a `to_int` function would divide by `SCALE` and truncate the fractional part. This would align the library with standard fixed-point APIs, reduce the risk of misuse, and lower the barrier to correct integration.

Update: *Resolved in pull request #264. The OpenZeppelin development team stated:*

> *We added corresponding convert modules to handle scaling from and to regular integer types. We didn't remove the casting API which is intentionally casting from raw bits with no scaling, and improve the documentation about both casting and converting APIs.*

M-02 Bitwise Operations on Signed Fixed-Point Type Expose Raw Two's Complement Encoding

The `SD29x9` signed fixed-point type uses two's complement encoding in a `u128`, where the most significant bit (bit 127) determines the sign. The library exposes seven bitwise operations

on this type: `and`, `and2`, `or`, `xor`, `not`, `lshift`, and `rshift`. With the exception of `rshift`, which performs an arithmetic right shift with sign extension, all of these operate directly on the raw `u128` bit pattern via `from_bits`, bypassing the sign-aware decomposition used by arithmetic functions.

Because the sign bit is not protected, any of these operations can silently flip, set, or clear it, turning a positive value negative or vice versa. For example, `or`-ing two positive `SD29x9` values whose magnitudes together set bit 127 would produce a negative result with no arithmetic justification. Similarly, `not` on any non-negative value always yields a negative value, and `lshift` can shift magnitude bits into the sign position. There is an internal inconsistency in how the type handles its encoding: arithmetic functions go through sign-magnitude decomposition, while bitwise operations bypass this and operate on the raw two's complement bits directly. This means integrators face two different mental models depending on which category of function they call.

Sui Move has no native signed integer type, so the two's complement encoding is purely an internal implementation detail of the library rather than a platform-level concept developers are expected to reason about. The expected utilization of bitwise operations on a decimal fixed-point type used for financial computation is low, while the potential for accidental sign corruption is high. While the existing documentation notes the raw-bit semantics, the function signatures alone do not distinguish these operations from the arithmetic API, and integrators working with decimal financial values may not expect bitwise helpers to alter the numeric sign of a value.

Consider removing these operations from `SD29x9`, or at minimum clearly isolating them (e.g., in a separate module or with explicit naming that signals raw-bit semantics). Integrators who genuinely need bitwise access to the raw bits can call `unwrap` to obtain the underlying `u128` and perform the operation explicitly, making the intent clear and the responsibility for sign handling visible at the call site.

Update: *Resolved in pull request #285. The OpenZeppelin development team stated:*

> *We've removed the operations as suggested.*

Low Severity

L-01 SD29x9 `pow` Uses Asymmetric Boundary Validation

`SD29x9` values are represented as two's-complement `u128` (`sd29x9.move`), and `pow` computes sign and magnitude separately using `decompose`, `res_neg` (`sd29x9_base.move`), and iterative magnitude updates. In this representation, `2^127` is reserved for the minimum negative value (`min()`), so positive outputs must remain strictly below that boundary.

Inside `pow`, the loop enforces `res_mag <= 2^127` through `assert!(res_mag <= MIN_NEGATIVE_VALUE as u256, EOverflow)`. However, final wrapping in `wrap_components` accepts `mag == 2^127` only for negative results (`value.neg && value.mag == min_negative`); otherwise it requires `value.mag < min_negative`. This creates an inclusive boundary check during iteration but an exclusive boundary check for final positive outputs.

Consider aligning these checks with the final sign domain, so intermediate validation uses `< 2^127` when `res_neg` is false and allows `<= 2^127` only when `res_neg` is true, or centralizing this rule in one shared helper used by both `pow` and `wrap_components`.

Update: *Resolved in pull request #280.*

L-02 Unsigned Fixed-Point Left Shift Silently Truncates on Overflow

The `UD30x9::lshift` function shifts the raw `u128` bits and silently discards any high bits that overflow, with no abort to signal data loss. An integrator using `lshift` as a multiplication-by-power-of-two could obtain a truncated result that is smaller than the input, with no indication that overflow occurred.

The function name `lshift` does not convey the unchecked truncation behavior. Consider renaming it to something more explicit (e.g., `unchecked_lshift` or `wrapping_lshift`) to signal that overflow is silently discarded. Additionally, providing a checked counterpart (e.g., `lshift` or `checked_lshift`) that aborts when non-zero bits would be shifted out of the

`u128` range would give integrators a safe default for cases where overflow represents a logic error. This naming convention would make the truncation behavior opt-in rather than implicit.

Update: *Resolved in pull request #288. The OpenZeppelin development team stated:*

> *We've applied the suggestions, including supporting both checked and unchecked right shift.*

L-03 Signed `mod` Function Name Does Not Reflect Truncated Remainder Semantics

The `SD29x9::mod` function computes the remainder by taking `abs(x) % abs(y)` and preserving the sign of the dividend, meaning a negative dividend produces a negative remainder (e.g., `mod(-7, 3)` returns `-1`). While this behavior is documented, the unqualified name `mod` creates a strong expectation of Euclidean modulo semantics, where the result is always non-negative, a property that is particularly natural for DeFi contexts such as index calculations or periodic boundary checks. Notably, Sui Move has no native signed integer type, so returning negative values from a `mod` operation may be unintuitive to developers on this platform.

Consider renaming the current function to something more explicit (e.g., `rem` or `truncated_mod`) to clearly communicate its truncated remainder semantics. Additionally, providing an `euclidean_mod` function that adjusts negative remainders by adding `abs(y)` to produce a non-negative result would give integrators the option that better fits most DeFi use cases. This approach preserves the existing functionality while reducing the risk of misuse from naming ambiguity.

Update: *Resolved in pull request #301. The OpenZeppelin development team stated:*

> *We've renamed the existing `mod` function to `rem` and added `mod` following Euclidean semantics.*

L-04 Fixed-Point `pow` Uses Linear Exponentiation, Accumulating Unnecessary Rounding Error

Both `SD29x9::pow` and `UD30x9::pow` compute `x^exp` by iterating `exp - 1` times, multiplying and truncating at each step (`result = result * base / SCALE`). This linear approach performs `O(exp)` multiplications, each of which introduces a truncation toward zero. The rounding error compounds with every iteration, and for fractional bases (`0 <`

`abs(x) < 1`), intermediate values can reach zero prematurely before the mathematically correct result would.

Binary exponentiation (exponentiation by squaring) would reduce the number of multiplications to `O(log(exp))`, producing fewer truncation steps and therefore less accumulated error. For example, computing `x^100` currently requires 99 multiply-and-truncate rounds, while binary exponentiation would require at most 7. The difference in precision is significant for higher exponents and is especially relevant for fractional bases where each truncation biases the result further toward zero.

Consider replacing the linear loop with a binary exponentiation algorithm. This would improve both gas efficiency and numerical accuracy. The sign derivation logic in `SD29x9::pow` (based on the parity of `exp`) would remain unchanged, as it is independent of the multiplication strategy.

Update: *Resolved in pull request #281. The OpenZeppelin development team stated:*

> *We've applied the suggestion.*

L-05 Decompose Is Not Publicly Exposed, Forcing Users to Validate Independently

In this fixed-point math library, value decomposition is handled by `decompose`, which is used throughout internal arithmetic and comparison flows (for example, `sd29x9_base.move` and `sd29x9_base.move`). This decomposition step is where the module normalizes sign/magnitude handling before downstream operations.

`decompose` is defined as an internal `fun` in `sd29x9_base.move`, and it is not exposed through the public surface in `sd29x9.move`. As a result, external users cannot call the canonical decomposition path directly and must reproduce equivalent validation logic in their own code, which can lead to inconsistent behavior across integrations.

Consider exposing a public helper that wraps `decompose` (or a controlled public package entry point) so callers can rely on one canonical decomposition and validation flow instead of reimplementing it externally.

Update: *Acknowledged, will resolve. The OpenZeppelin development team stated:*

> *Adding public `decompose` function isn't difficult, but needs careful consideration and some time to properly design it. We will consider adding it in the following releases.*

L-06 Missing Rounding-Direction Variants for Multiplication and Division

The `UD30x9::mul`, `UD30x9::div`, `SD29x9::mul`, and `SD29x9::div` functions only truncate toward zero when the exact result cannot be represented. DeFi protocols frequently require explicit control over rounding direction to maintain invariants,for example, rounding against the user when computing fees or token redemption amounts to prevent value extraction. Without built-in rounding variants, integrators must compose operations manually (e.g., applying `ceil` after `mul`), which incurs unnecessary overhead and increases the risk of incorrect rounding logic.

Consider adding rounding-up variants (e.g., `mul_up`, `div_up`) that round away from zero when a remainder exists, and renaming the current truncating versions to explicitly reflect their rounding direction (e.g., `mul_down`, `div_down`). This naming convention makes the rounding behavior opt-in and visible at every call site, reducing the risk of integrators unknowingly relying on a default truncation direction. Providing both variants is a well-established pattern in fixed-point math libraries targeting DeFi.

Update: *Partially Resolved in pull request #289. The OpenZeppelin development team stated:*

> *We've added trunc and away variants. Didn't directly rename the core names mul and div.*

Notes & Additional Information

N-01 Redundant Bitmask in `SD29x9::lshift`

The `SD29x9::lshift` function applies a `& U128_MAX_VALUE` mask to the result of `x.unwrap() << bits`. Since `unwrap` returns a `u128`, the left shift already produces a `u128` and cannot exceed 128 bits. The bitmask has no effect.

Consider removing the `& U128_MAX_VALUE` operation to improve readability and avoid implying that overflow truncation is being handled when it is not.

Update: *Resolved in pull request #285. The OpenZeppelin development team stated:*

> *Function has been removed as part of M-02.*

N-02 Division-by-Zero Check in `mod` and `div` Relies on VM Runtime Abort

The `mod` and `div` functions in both `SD29x9` and `UD30x9` document that they abort when the divisor is zero, but none of them include an explicit check. The abort is instead produced by the Move VM's runtime division-by-zero error, which yields a generic error code rather than a descriptive, library-defined one.

Consider adding explicit zero-divisor assertions with custom error constants, consistent with how other functions in the library (e.g., `pow`, `wrap`) handle invalid inputs. This would provide integrators with actionable error messages and make the documented behavior match an intentional code path rather than an implicit VM guarantee.

Update: *Resolved in pull request #289. The OpenZeppelin development team stated:*

> *We've added division checks.*

N-03 `UD30x9::sub` Uses Overflow Error Constant for Underflow Condition

The `UD30x9::sub` function checks that `x >= y` before computing the difference, and aborts with `EOverflow` when the `y` exceeds the `x`. This condition is an arithmetic underflow (the result would be negative, which is unrepresentable in an unsigned type), not an overflow.

Consider introducing a distinct `EUnderflow` error constant for this case, or at minimum updating the error message string to accurately describe the condition. Reusing `EOverflow` here is misleading to integrators debugging failed transactions, as the error message suggests the result exceeded the upper bound when the actual cause is a negative result.

Update: *Resolved in pull request #297. The OpenZeppelin development team stated:*

> *We've added the suggested distinct `EUnderflow` error code.*

N-04 Inconsistent `SCALE` Constant Type Between SD29x9 Modules

The `SD29x9` implementation is split between a type definition module (`sd29x9.move`) and an arithmetic module (`sd29x9_base.move`). The same `SCALE` value (`1_000_000_000`) is declared as `u128` in `sd29x9.move` and as `u256` in `sd29x9_base.move`, both under the name `SCALE`. While this is valid since Move constants are module-scoped, the identical name with different types can mislead developers adding new code paths who may assume `SCALE` is consistently `u128` across the SD29x9 modules.

The `UD30x9` modules already handle this by using distinct names: `SCALE` (`u128`) and `SCALE_U256` (`u256`). Consider adopting the same convention in the SD29x9 modules for internal consistency across the library.

Update: *Resolved in pull request #264. The OpenZeppelin development team stated:*

> *We've fixed this as part of the constants refactor.*

N-05 Missing Rounding Direction Documentation in `SD29x9` Arithmetic Functions

The `SD29x9::mul` and `SD29x9::div` functions do not document their rounding behavior. Both functions decompose operands into sign and magnitude, perform the operation on magnitudes using integer division (which truncates toward zero), and reapply the sign. This means results are rounded toward zero regardless of sign. However, the documentation for `mul` only states it returns "The product `x * y`" and `div` only states it returns "The division result `x / y`", with no mention of rounding.

The unsigned counterparts `UD30x9::mul` and `UD30x9::div` do document this, explicitly noting that the result is "rounded down toward zero." Rounding direction is especially important for the signed type, where "toward zero" and "toward negative infinity" produce different results for negative values.

Additionally, the `SD29x9::unchecked_add` and `SD29x9::unchecked_sub` functions document their return values as "The result of raw bits addition/subtraction" without specifying wrapping modulo `2^128` semantics, while their `UD30x9` counterparts explicitly document the wrapping behavior.

Consider aligning the SD29x9 documentation with the UD30x9 documentation by adding rounding direction to `mul` and `div`, and wrapping semantics to `unchecked_add` and `unchecked_sub`.

Update: *Resolved in [pull request #289](). The OpenZeppelin development team stated:*

> *We've updated the documentation.*

Conclusion

The `openzeppelin_fp_math` library provides a focused and self-contained fixed-point arithmetic foundation for the Sui ecosystem. By offering both unsigned and signed decimal types with 9 decimal places of precision, matching Sui's native coin decimals, the library targets a core need for DeFi protocols that require exact decimal computation without floating-point approximation. The two-module architecture that separates type definitions from arithmetic operations results in a clean API surface that is straightforward to integrate.

The audit identified two medium-severity and six low-severity issues, along with five informational findings. The medium-severity findings relate to API surface gaps and unsafe patterns that increase misuse potential for integrators. The low-severity findings address overflow behavior, validation inconsistencies, and semantic mismatches in function naming and behavior. The informational findings cover documentation gaps, redundant operations, and internal consistency.

Overall, the codebase follows a consistent structure across the unsigned and signed variants, and the separation of concerns between type definition modules and arithmetic modules supports clear auditability.

The OpenZeppelin development team is commended for their responsiveness and willingness to provide detailed technical context throughout the engagement.

Appendix

Issue Classification

OpenZeppelin classifies smart contract vulnerabilities on a 5-level scale:

- Critical
- High
- Medium
- Low
- Note/Information

Critical Severity

This classification is applied when the issue's impact is catastrophic, threatening extensive damage to the client's reputation and/or causing severe financial loss to the client or users. The likelihood of exploitation can be high, warranting a swift response. Critical issues typically involve significant risks such as the permanent loss or locking of a large volume of users' sensitive assets or the failure of core system functionalities without viable mitigations. These issues demand immediate attention due to their potential to compromise system integrity or user trust significantly.

High Severity

These issues are characterized by the potential to substantially impact the client's reputation and/or result in considerable financial losses. The likelihood of exploitation is significant, warranting a swift response. Such issues might include temporary loss or locking of a significant number of users' sensitive assets or disruptions to critical system functionalities, albeit with potential, yet limited, mitigations available. The emphasis is on the significant but not always catastrophic effects on system operation or asset security, necessitating prompt and effective remediation.

Medium Severity

Issues classified as being of medium severity can lead to a noticeable negative impact on the client's reputation and/or moderate financial losses. Such issues, if left unattended, have a moderate likelihood of being exploited or may cause unwanted side effects in the system.

These issues are typically confined to a smaller subset of users' sensitive assets or might involve deviations from the specified system design that, while not directly financial in nature, compromise system integrity or user experience. The focus here is on issues that pose a real but contained risk, warranting timely attention to prevent escalation.

Low Severity

Low-severity issues are those that have a low impact on the client's operations and/or reputation. These issues may represent minor risks or inefficiencies to the client's specific business model. They are identified as areas for improvement that, while not urgent, could enhance the security and quality of the codebase if addressed.

Notes & Additional Information Severity

This category is reserved for issues that, despite having a minimal impact, are still important to resolve. Addressing these issues contributes to the overall security posture and code quality improvement but does not require immediate action. It reflects a commitment to maintaining high standards and continuous improvement, even in areas that do not pose immediate risks.